```
            is_bold = true
            start_block = index + 1
        ## Handle end of bold text
        ## Remember that the last number in a slice is the index
        ## after the last index used.
        if text[index] == "</B>":
            if not is_bold:
                print("Error: Extra Close Bold")
            print("Bold [", start_block, ":", index, "]",
 text[start_block:index])
            is_bold = false
            start_block = index + 1

get_bold(poem)
```

with the output being:

```
Bold [ 1 : 4 ] ['Jack', 'and', 'Jill']
Bold [ 11 : 15 ] ['fetch', 'a', 'pail', 'of']
Bold [ 20 : 23 ] ['down', 'and', 'broke']
Bold [ 28 : 30 ] ['Jill', 'came']
```

The `get_bold()` function takes in a list that is broken into words and tokens. The tokens that it looks for are <B> which starts the bold text and </B> which ends bold text. The function `get_bold()` goes through and searches for the start and end tokens.

The next feature of lists is copying them. If you try something simple like:

```
>>> a = [1, 2, 3]
»> b = a
»> print(b)
[1, 2, 3]
»> b[1] = 10
»> print(b)
[1, 10, 3]
»> print(a)
[1, 10, 3]
```

This probably looks surprising since a modification to `b`resulted in `a` being changed as well. What happened is that the statement `b = a` makes `b` a *reference* to `a`. This means that `b` can be thought of as another name for `a`. Hence any modification to `b` changes `a` as well. However some assignments don't create two names for one list:

```
>>> a = [1, 2, 3]
»> b = a * 2
»> print(a)
[1, 2, 3]
»> print(b)
[1, 2, 3, 1, 2, 3]
»> a[1] = 10
»> print(a)
[1, 10, 3]
»> print(b)
[1, 2, 3, 1, 2, 3]
```

In this case `b` is not a reference to `a` since the expression `a * 2` creates a new list. Then the statement `b = a * 2` gives `b` a reference to `a * 2` rather than a reference to `a`. All assignment operations create a reference. When you pass a list as an argument to a function you create a reference as well. Most of the time you don't have to worry about creating references rather than copies. However when you need to make modifications to one list without changing another name of the list you have to make sure that you have actually created a copy.

There are several ways to make a copy of a list. The simplest that works most of the time is the slice operator since it always makes a new list even if it is a slice of a whole list:

```
>>> a = [1, 2, 3]
>> b = a[:]
>> b[1] = 10
>> print(a)
[1, 2, 3]
>> print(b)
[1, 10, 3]
```

Taking the slice `[:]` creates a new copy of the list. However it only copies the outer list. Any sublist inside is still a references to the sublist in the original list. Therefore, when the list contains lists, the inner lists have to be copied as well. You could do that manually but Python already contains a module to do it. You use the `deepcopy` function of the `copy` module:

```
>>> import copy
>> a = [[1, 2, 3], [4, 5, 6]]
>> b = a[:]
>> c = copy.deepcopy(a)
>> b[0][1] = 10
>> c[1][1] = 12
>> print(a)
[[1, 10, 3], [4, 5, 6]]
>> print(b)
[[1, 10, 3], [4, 5, 6]]
>> print(c)
[[1, 2, 3], [4, 12, 6]]
```

First of all notice that `a` is a list of lists. Then notice that when `b[0][1] = 10` is run both `a` and `b` are changed, but `c` is not. This happens because the inner arrays are still references when the slice operator is used. However with `deepcopy` `c` was fully copied.

So, should I worry about references every time I use a function or `=`? The good news is that you only have to worry about references when using dictionaries and lists. Numbers and strings create references when assigned but every operation on numbers and strings that modifies them creates a new copy so you can never modify them unexpectedly. You do have to think about references when you are modifying a list or a dictionary.

By now you are probably wondering why are references used at all? The basic reason is speed. It is much faster to make a reference to a thousand element list than to copy all the elements. The other reason is that it allows you to have a function to modify the inputted list or dictionary. Just remember about references if you ever have some weird problem with data being changed when it shouldn't be.

16 Revenge of the Strings

And now presenting a cool trick that can be done with strings:

```
def shout(string):
    for character in string:
        print("Gimme an " + character)
        print("'" + character + "'")

shout("Lose")

def middle(string):
    print("The middle character is:", string[len(string) // 2])

middle("abcdefg")
middle("The Python Programming Language")
middle("Atlanta")
```

And the output is:

```
 Gimme an L
 'L'
 Gimme an o
 'o'
 Gimme an s
 's'
 Gimme an e
 'e'
 The middle character is: d
 The middle character is: r
 The middle character is: a
```

What these programs demonstrate is that strings are similar to lists in several ways. The `shout()` function shows that `for` loops can be used with strings just as they can be used with lists. The `middle` procedure shows that that strings can also use the `len()` function and array indexes and slices. Most list features work on strings as well.

The next feature demonstrates some string specific features:

```
def to_upper(string):
    ## Converts a string to upper case
    upper_case = ""
    for character in string:
        if 'a' <= character <= 'z':
            location = ord(character) - ord('a')
            new_ascii = location + ord('A')
            character = chr(new_ascii)
        upper_case = upper_case + character
    return upper_case

print(to_upper("This is Text"))
```

with the output being:

```
THIS IS TEXT
```

This works because the computer represents the characters of a string as numbers from 0 to 1,114,111. For example 'A' is 65, 'B' is 66 and ℵis 1488. The values are the unicode[1] value. Python has a function called `ord()` (short for ordinal) that returns a character as a number. There is also a corresponding function called `chr()` that converts a number into a character. With this in mind the program should start to be clear. The first detail is the line: `if 'a' <= character <= 'z':` which checks to see if a letter is lower case. If it is then the next lines are used. First it is converted into a location so that a = 0, b = 1, c = 2 and so on with the line: `location = ord(character) - ord('a')`. Next the new value is found with `new_ascii = location + ord('A')`. This value is converted back to a character that is now upper case. Note that if you really need the upper case of a letter, you should use `u=var.upper()` which will work with other languages as well.

Now for some interactive typing exercise:

```
>>> # Integer to String
»> 2
2
»> repr(2)
'2'
»> -123
-123
»> repr(-123)
'-123'
»> # String to Integer
»> "23"
'23'
»> int("23")
23
»> "23" * 2
'2323'
»> int("23") * 2
46
»> # Float to String
»> 1.23
1.23
»> repr(1.23)
'1.23'
»> # Float to Integer
»> 1.23
1.23
»> int(1.23)
1
»> int(-1.23)
-1
»> # String to Float
»> float("1.23")
1.23
»> "1.23"
'1.23'
»> float("123")
123.0
```

1 https://en.wikipedia.org/wiki/unicode

If you haven't guessed already the function `repr()` can convert an integer to a string and the function `int()` can convert a string to an integer. The function `float()` can convert a string to a float. The `repr()` function returns a printable representation of something. Here are some examples of this:

```
>>> repr(1)
'1'
>>> repr(234.14)
'234.14'
>>> repr([4, 42, 10])
'[4, 42, 10]'
```

The `int()` function tries to convert a string (or a float) into an integer. There is also a similar function called `float()` that will convert an integer or a string into a float. Another function that Python has is the `eval()` function. The `eval()` function takes a string and returns data of the type that python thinks it found. For example:

```
>>> v = eval('123')
>>> print(v, type(v))
123 <type 'int'>
>>> v = eval('645.123')
>>> print(v, type(v))
645.123 <type 'float'>
>>> v = eval('[1, 2, 3]')
>>> print(v, type(v))
[1, 2, 3] <type 'list'>
```

If you use the `eval()` function you should check that it returns the type that you expect.

One useful string function is the `split()` method. Here's an example:

```
>>> "This is a bunch of words".split()
['This', 'is', 'a', 'bunch', 'of', 'words']
>>> text = "First batch, second batch, third, fourth"
>>> text.split(",")
['First batch', ' second batch', ' third', ' fourth']
```

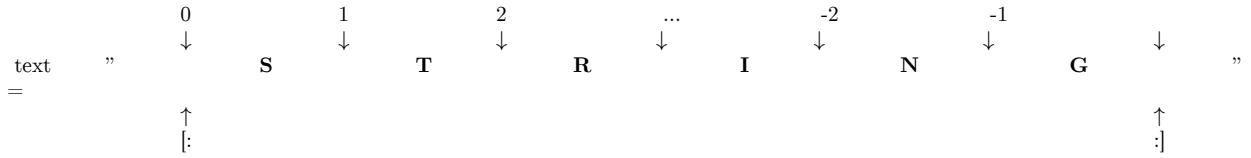
Notice how `split()` converts a string into a list of strings. The string is split by whitespace by default or by the optional argument (in this case a comma). You can also add another argument that tells `split()` how many times the separator will be used to split the text. For example:

```
>>> list = text.split(",")
>>> len(list)
4
>>> list[-1]
' fourth'
>>> list = text.split(",", 2)
>>> len(list)
3
>>> list[-1]
' third, fourth'
```

16.0.1 Slicing strings (and lists)

Strings can be cut into pieces — in the same way as it was shown for lists in the previous chapter — by using the *slicing* "operator" `[]`. The slicing operator works in the same way as before: text[first_index:last_index] (in very rare cases there can be another colon and a third argument, as in the example shown below).

In order not to get confused by the index numbers, it is easiest to see them as *clipping places*, possibilities to cut a string into parts. Here is an example, which shows the clipping places (in yellow) and their index numbers (red and blue) for a simple text string:



Note that the red indexes are counted from the beginning of the string and the blue ones from the end of the string backwards. (Note that there is no blue -0, which could seem to be logical at the end of the string. Because `-0 == 0`, -0 means "beginning of the string" as well.) Now we are ready to use the indexes for slicing operations:

text[1:4]	→	"TRI"
text[:5]	→	"STRIN"
text[:-1]	→	"STRIN"
text[-4:]	→	"RING"
text[2]	→	"R"
text[:]	→	"STRING"
text[::-1]	→	"GNIRTS"

`text[1:4]` gives us all of the `text` string between clipping places 1 and 4, "`TRI`". If you omit one of the [first_index:last_index] arguments, you get the beginning or end of the string as default: `text[:5]` gives "`STRIN`". For both `first_index` and `last_index` we can use both the red and the blue numbering schema: `text[:-1]` gives the same as `text[:5]` , because the index -1 is at the same place as 5 in this case. If we do not use an argument containing a colon, the number is treated in a different way: `text[2]` gives us one character following the second clipping point, "`R`". The special slicing operation `text[:]` means "from the beginning to the end" and produces a copy of the entire string (or list, as shown in the previous chapter).

Last but not least, the slicing operation can have a second colon and a third argument, which is interpreted as the "step size": `text[::-1]` is `text` from beginning to the end, with a step size of -1. -1 means "every character, but in the other direction". "`STRING`" backwards is "`GNIRTS`" (test a step length of 2, if you have not got the point here).

All these slicing operations work with lists as well. In that sense strings are just a special case of lists, where the list elements are single characters. Just remember the concept of *clipping places*, and the indexes for slicing things will get a lot less confusing.